

CS3301 - Data Structures

Topic: insertion sort

1. SUMMARY

Insertion sort is a simple sorting algorithm that works by dividing the input into a sorted and an unsorted region. Each subsequent element is inserted into the sorted region in its correct position. The time complexity of insertion sort is $O(n^2)$ in the worst case, where n is the number of elements to be sorted. The space complexity is $O(1)$, as it only requires a small amount of additional memory to store the current element being inserted. Insertion sort has several applications, including sorting small lists of integers or strings. It is also used in some specialized algorithms, such as the merge sort algorithm.

The reference discusses insertion sort as a key principle of data structures and algorithms. It highlights the importance of selecting the correct data structure and algorithm to efficiently solve a problem. The reference also mentions the use of insertion sort in various applications, including sorting small lists of integers or strings.

Insertion sort is a stable sorting algorithm, meaning that the order of equal elements is preserved. It is also a comparison-based sorting algorithm, meaning that it compares elements to determine their correct position in the sorted region. The reference provides a detailed explanation of the working principle of insertion sort, including step-by-step examples and diagrams.

2. SHORT NOTES

Detailed Definition with Examples

Insertion sort is a simple sorting algorithm that works by dividing the input into a sorted and an unsorted region. Each subsequent element is inserted into the sorted region in its correct position. The algorithm starts with the first element as the sorted region and then iterates through the remaining elements, inserting each one into the correct position in the sorted region.

For example, consider the following list of integers: [5, 3, 8, 2, 9, 1]. The insertion sort algorithm would first consider the first element (5) as the sorted region and then insert the remaining elements into their correct position. The sorted region would be [1, 2, 3, 5, 8, 9].

Types and Classifications

Insertion sort is a stable sorting algorithm, meaning that the order of equal elements is preserved. It is also a comparison-based sorting algorithm, meaning that it compares elements to determine their correct position in the sorted region.

There are several types of insertion sort algorithms, including:

- * Simple insertion sort: This is the most basic form of insertion sort, where each element is inserted into the correct position in the sorted region.
- * Insertion sort with a key: This type of insertion sort uses a key to determine the correct position of each element in the sorted region.
- * Insertion sort with a binary search: This type of insertion sort uses a binary search to find the correct position of each element in the sorted region.

Step-by-Step Working Principle

The working principle of insertion sort can be explained as follows:

1. Start with the first element as the sorted region.
2. Iterate through the remaining elements, one by one.
3. For each element, compare it to the corresponding element in the sorted region.
4. If the element is less than the corresponding element, insert it into the correct position in the sorted region.
5. If the element is greater than the corresponding element, move to the next element in the sorted region.
6. Repeat steps 3-5 until all elements have been inserted into the sorted region.

Important Formulas and Equations

The time complexity of insertion sort is $O(n^2)$ in the worst case, where n is the number of elements to be sorted. The space complexity is $O(1)$, as it only requires a small amount of additional memory to store the

current element being inserted.

The formula for the time complexity of insertion sort is:

$$T(n) = n^2$$

where $T(n)$ is the time complexity and n is the number of elements to be sorted.

Diagrams Description

The following diagram illustrates the working principle of insertion sort:

[Insertion Sort Diagram]

In this diagram, the sorted region is represented by the shaded area, and the unsorted region is represented by the unshaded area. Each element in the unsorted region is inserted into the correct position in the sorted region, as explained in the step-by-step working principle.

Advantages and Disadvantages

The advantages of insertion sort include:

- * Simple to implement
- * Stable sorting algorithm
- * Comparison-based sorting algorithm

The disadvantages of insertion sort include:

- * Time complexity of $O(n^2)$ in the worst case
- * Not suitable for large datasets

Real-World Applications

Insertion sort has several real-world applications, including:

- * Sorting small lists of integers or strings
- * Using in some specialized algorithms, such as the merge sort algorithm

Comparison with Related Concepts

Insertion sort can be compared to other sorting algorithms, such as:

- * Bubble sort: This algorithm is similar to insertion sort, but it repeatedly iterates through the list, comparing adjacent elements and swapping them if they are in the wrong order.
- * Selection sort: This algorithm is similar to insertion sort, but it repeatedly selects the smallest element from the unsorted region and inserts it into the sorted region.

Time and Space Complexity

The time complexity of insertion sort is $O(n^2)$ in the worst case, where n is the number of elements to be sorted. The space complexity is $O(1)$, as it only requires a small amount of additional memory to store the current element being inserted.

3. CODING EXAMPLES

C Implementation

```
```c
```

```
#include <stdio.h>
```

```
void insertion_sort(int arr[], int n) {
 for (int i = 1; i < n; i++) {
 int key = arr[i];
 int j = i - 1;
 while (j >= 0 && arr[j] > key) {
 arr[j + 1] = arr[j];
```

```
 j--;\n }\n arr[j + 1] = key;\n}\n}
```

```
int main() {\n int arr[] = {5, 3, 8, 2, 9, 1};\n int n = sizeof(arr) / sizeof(arr[0]);\n insertion_sort(arr, n);\n for (int i = 0; i < n; i++) {\n printf("%d ", arr[i]);\n }\n return 0;\n}\n```\n
```

### Python Implementation

```
```python\n\ndef insertion_sort(arr):\n    for i in range(1, len(arr)):\n        key = arr[i]\n        j = i - 1\n        while j >= 0 and arr[j] > key:\n            arr[j + 1] = arr[j]\n            j -= 1\n        arr[j + 1] = key\n    return arr\n\narr = [5, 3, 8, 2, 9, 1]\nprint(insertion_sort(arr))\n```\n
```

...

Algorithm

Step 1: Start with the first element as the sorted region.

Step 2: Iterate through the remaining elements, one by one.

Step 3: For each element, compare it to the corresponding element in the sorted region.

Step 4: If the element is less than the corresponding element, insert it into the correct position in the sorted region.

Step 5: If the element is greater than the corresponding element, move to the next element in the sorted region.

Step 6: Repeat steps 3-5 until all elements have been inserted into the sorted region.

4. IMPORTANT QUESTIONS

Q1. (2 marks) What is the time complexity of insertion sort in the worst case?

Answer: The time complexity of insertion sort is $O(n^2)$ in the worst case, where n is the number of elements to be sorted.

Q2. (2 marks) What is the space complexity of insertion sort?

Answer: The space complexity of insertion sort is $O(1)$, as it only requires a small amount of additional memory to store the current element being inserted.

Q3. (13 marks) Explain the working principle of insertion sort.

Answer: The working principle of insertion sort can be explained as follows: Start with the first element as the sorted region. Iterate through the remaining elements, one by one. For each element, compare it to the corresponding element in the sorted region. If the element is less than the corresponding element, insert it into the correct position in the sorted region. If the element is greater than the corresponding element, move to the next element in the sorted region. Repeat steps 3-5 until all elements have been inserted into the sorted region.

Q4. (13 marks) What are the advantages and disadvantages of insertion sort?

Answer: The advantages of insertion sort include: Simple to implement, stable sorting algorithm, comparison-based sorting algorithm. The disadvantages of insertion sort include: Time complexity of $O(n^2)$ in the worst case, not suitable for large datasets.

Q5. (13 marks) Compare insertion sort with other sorting algorithms.

Answer: Insertion sort can be compared to other sorting algorithms, such as bubble sort and selection sort. Bubble sort is similar to insertion sort, but it repeatedly iterates through the list, comparing adjacent elements and swapping them if they are in the wrong order. Selection sort is similar to insertion sort, but it repeatedly selects the smallest element from the unsorted region and inserts it into the sorted region.

Q6. (15 marks) Explain the real-world applications of insertion sort.

Answer: Insertion sort has several real-world applications, including: Sorting small lists of integers or strings, using in some specialized algorithms, such as the merge sort algorithm.

Q7. (15 marks) Explain the time and space complexity of insertion sort.

Answer: The time complexity of insertion sort is $O(n^2)$ in the worst case, where n is the number of elements to be sorted. The space complexity is $O(1)$, as it only requires a small amount of additional memory to store the current element being inserted.

Q8. (15 marks) Explain the diagram of insertion sort.

Answer: The following diagram illustrates the working principle of insertion sort: [Insertion Sort Diagram]. In this diagram, the sorted region is represented by the shaded area, and the unsorted region is represented by the unshaded area. Each element in the unsorted region is inserted into the correct position in the sorted region, as explained in the step-by-step working principle.

Q9. (2 marks) What is the formula for the time complexity of insertion sort?

Answer: The formula for the time complexity of insertion sort is $T(n) = n^2$, where $T(n)$ is the time complexity and n is the number of elements to be sorted.

Q10. (2 marks) What is the space complexity of insertion sort?

Answer: The space complexity of insertion sort is $O(1)$, as it only requires a small amount of additional memory to store the current element being inserted.

Q11. (2 marks) What is the working principle of insertion sort?

Answer: The working principle of insertion sort can be explained as follows: Start with the first element as the sorted region. Iterate through the remaining elements, one by one. For each element, compare it to the corresponding element in the sorted region. If the element is less than the corresponding element, insert it into the correct position in the sorted region. If the element is greater than the corresponding element, move to the next element in the sorted region. Repeat steps 3-5 until all elements have been inserted into the sorted region.

Q12. (2 marks) What are the advantages of insertion sort?

Answer: The advantages of insertion sort include: Simple to implement, stable sorting algorithm, comparison-based sorting algorithm.

Q13. (2 marks) What are the disadvantages of insertion sort?

Answer: The disadvantages of insertion sort include: Time complexity of $O(n^2)$ in the worst case, not suitable for large datasets.

Q14. (2 marks) What is the real-world application of insertion sort?

Answer: Insertion sort has several real-world applications, including: Sorting small lists of integers or strings, using in some specialized algorithms, such as the merge sort algorithm.

Q15. (2 marks) What is the time complexity of insertion sort?

Answer: The time complexity of insertion sort is $O(n^2)$ in the worst case, where n is the number of elements to be sorted.

Q1. What is the time complexity of insertion sort in the worst case?

- a) $O(n)$
- b) $O(n^2)$
- c) $O(n \log n)$
- d) $O(1)$

Answer: b) $O(n^2)$

Explanation: The time complexity of insertion sort is $O(n^2)$ in the worst case, where n is the number of elements to be sorted.

Q2. What is the space complexity of insertion sort?

- a) $O(1)$
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n \log n)$

Answer: a) $O(1)$

Explanation: The space complexity of insertion sort is $O(1)$, as it only requires a small amount of additional memory to store the current element being inserted.

Q3. What is the working principle of insertion sort?

- a) Start with the first element as the sorted region, and then insert each subsequent element into the correct position.
- b) Start with the last element as the sorted region, and then insert each subsequent element into the correct position.
- c) Start with the first element as the unsorted region, and then insert each subsequent element into the correct position.
- d) Start with the last element as the unsorted region, and then insert each subsequent element into the

correct position.

Answer: a) Start with the first element as the sorted region, and then insert each subsequent element into the correct position.

Explanation: The working principle of insertion sort is to start with the first element as the sorted region, and then insert each subsequent element into the correct position.

Q4. What are the advantages of insertion sort?

- a) Simple to implement, stable sorting algorithm, comparison-based sorting algorithm.
- b) Fast, efficient, and suitable for large datasets.
- c) Simple to implement, but not stable or comparison-based.
- d) Fast and efficient, but not suitable for small datasets.

Answer: a) Simple to implement, stable sorting algorithm, comparison-based sorting algorithm.

Explanation: The advantages of insertion sort include: Simple to implement, stable sorting algorithm, comparison-based sorting algorithm.

Q5. What are the disadvantages of insertion sort?

- a) Time complexity of $O(n^2)$ in the worst case, not suitable for large datasets.
- b) Fast and efficient, but not stable or comparison-based.
- c) Simple to implement, but not fast or efficient.
- d) Not suitable for small datasets.

Answer: a) Time complexity of $O(n^2)$ in the worst case, not suitable for large datasets.

Explanation: The disadvantages of insertion sort include: Time complexity of $O(n^2)$ in the worst case, not suitable for large datasets.

Q6. What is the real-world application of insertion sort?

- a) Sorting small lists of integers or strings.
- b) Using in some specialized algorithms, such as the merge sort algorithm.
- c) Both a and b.
- d) Neither a nor b.

Answer: c) Both a and b.

Explanation: Insertion sort has several real-world applications, including: Sorting small lists of integers or strings, using in some specialized algorithms, such as the merge sort algorithm.

Q7. What is the time complexity of insertion sort?

- a) $O(n)$
- b) $O(n^2)$
- c) $O(n \log n)$
- d) $O(1)$

Answer: b) $O(n^2)$

Explanation: The time complexity of insertion sort is $O(n^2)$ in the worst case, where n is the number of elements to be sorted.

Q8. What is the space complexity of insertion sort?

- a) $O(1)$
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n \log n)$

Answer: a) $O(1)$

Explanation: The space complexity of insertion sort is $O(1)$, as it only requires a small amount of additional memory to store the current element being inserted.

Q9. What is the working principle of insertion sort?

- a) Start with the first element as the sorted region, and then insert each subsequent element into the correct position.
- b) Start with the last element as the sorted region, and then insert each subsequent element into the correct position.
- c) Start with the first element as the unsorted region, and then insert each subsequent element into the correct position.
- d) Start with the last element as the unsorted region, and then insert each subsequent element into the correct position.

Answer: a) Start with the first element as the sorted region, and then insert each subsequent element into the correct position.

Explanation: The working principle of insertion sort is to start with the first element as the sorted region, and then insert each subsequent element into the correct position.

Q10. What are the advantages of insertion sort?

- a) Simple to implement, stable sorting algorithm, comparison-based sorting algorithm.
- b) Fast, efficient, and suitable for large datasets.
- c) Simple to implement, but not stable or comparison-based.
- d) Fast and efficient, but not suitable for small datasets.

Answer: a) Simple to implement, stable sorting algorithm, comparison-based sorting algorithm.

Explanation: The advantages of insertion sort include: Simple to implement, stable sorting algorithm, comparison-based sorting algorithm.

Q11. What are the disadvantages of insertion sort?

- a) Time complexity of $O(n^2)$ in the worst case, not suitable for large datasets.
- b) Fast and efficient, but not stable or comparison-based.

- c) Simple to implement, but not fast or efficient.
- d) Not suitable for small datasets.

Answer: a) Time complexity of $O(n^2)$ in the worst case, not suitable for large datasets.

Explanation: The disadvantages of insertion sort include: Time complexity of $O(n^2)$ in the worst case, not suitable for large datasets.

Q12. What is the real-world application of insertion sort?

- a) Sorting small lists of integers or strings.
- b) Using in some specialized algorithms, such as the merge sort algorithm.
- c) Both a and b.
- d) Neither a nor b.

Answer: c) Both a and b.

Explanation: Insertion sort has several real-world applications, including: Sorting small lists of integers or strings, using in some specialized algorithms, such as the merge sort algorithm.

Q13. What is the time complexity of insertion sort?

- a) $O(n)$
- b) $O(n^2)$
- c) $O(n \log n)$
- d) $O(1)$

Answer: b) $O(n^2)$

Explanation: The time complexity of insertion sort is $O(n^2)$ in the worst case, where n is the number of elements to be sorted.

Q14. What is the space complexity of insertion sort?

- a) $O(1)$
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n \log n)$

Answer: a) $O(1)$

Explanation: The space complexity of insertion sort is $O(1)$, as it only requires a small amount of additional memory to store the current element being inserted.

Q15. What is the working principle of insertion sort?

- a) Start with the first element as the sorted region, and then insert each subsequent element into the correct position.
- b) Start with the last element as the sorted region, and then insert each subsequent element into the correct position.
- c) Start with the first element as the unsorted region, and then insert each subsequent element into the correct position.
- d) Start with the last element as the unsorted region, and then insert each subsequent element into the correct position.

Answer: a) Start with the first element as the sorted region, and then insert each subsequent element into the correct position.

Explanation: The working principle of insertion sort is to start with the first element as the sorted region, and then insert each subsequent element into the correct position.